Wireshark Sample Capture Writeup 2

`telnet-cooked.pcap`

Week 2, DG Interns Hub Cyber Security Internship, Network Traffic Analysis

Prepared by: **Harini**

Summary. I analysed this capture to answer three questions: which protocols appear, what happened, and whether anything is suspicious.
Protocols: Ethernet, IPv4, TCP and Telnet.
What happened: a user at 192.168.0.2 connected to a server at 192.168.0.1 with Telnet, logged in, ran a ping and two file listings, and then logged out.
Suspicious: I found no sign of an attack. The main weakness is Telnet itself, because the login name, the password and every command travel as readable text.

1. About the capture

Item	Detail
Source	Wireshark SampleCaptures wiki, Telnet section. The wiki describes it as a telnet session in cooked (per-line) mode.
Tool	Wireshark 4.6.9 on Windows 11.
Size	92 packets, all in one TCP conversation (stream 0).
Hosts	Client: 192.168.0.2, port 1550. Server: 192.168.0.1, port 23, the standard Telnet port.
Age of the traffic	The server banner shows a kernel build date of 12 Oct 1999, so this is old traffic.

I opened the file in Wireshark and used these tools: Statistics, Protocol Hierarchy, Follow TCP Stream, and the display filter `_ws.malformed`.

2. Which protocols appear

I opened Statistics, Protocol Hierarchy to count the protocols (Figure 1).

Protocol	Packets	What it does in this capture
Ethernet and IPv4	92 (100%)	They carry every packet.
TCP	92 (100%)	It gives reliable transport. The first three packets are the handshake (SYN, SYN-ACK, ACK) from 192.168.0.2 to port 23 of 192.168.0.1.
Telnet	46 (50.0%)	It carries the remote terminal session: the option negotiation, the login, and the commands and their output. These packets hold 1634 bytes.
Malformed Packet	1 (1.1%)	One Telnet packet that Wireshark could not read fully. I explain it in Section 4.

The other 46 packets are TCP only, such as the handshake and the acknowledgements. No DNS or other protocols appear, because the client connected straight to an IP address.

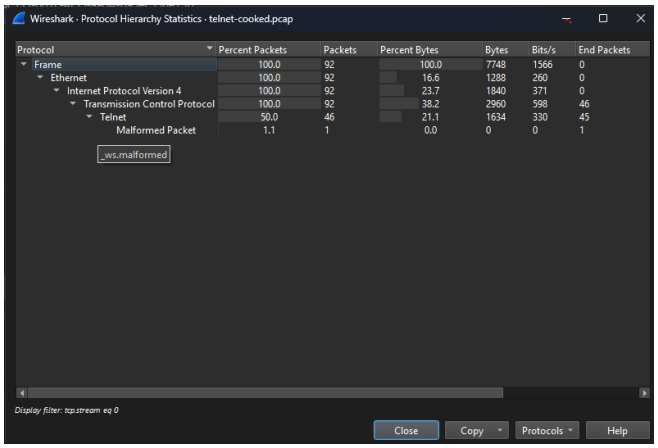


Figure 1. Protocol Hierarchy of the capture (92 packets).

3. What happened

I followed the session in order. Here is the story in four steps.

Step 1. The connection and the Telnet options

The client 192.168.0.2 opened a TCP connection to port 23 of 192.168.0.1 (packets 1 to 3). From packet 4, the two sides exchanged Telnet options (Figure 2). The server asked for an authentication option, and the client answered Won't Authentication Option (packets 5 and 7). Later the client also answered Don't Encryption Option and Won't Encryption Option (packet 13). So the session used no Telnet authentication option and no encryption option. The sides also exchanged window size, terminal speed and X display location.

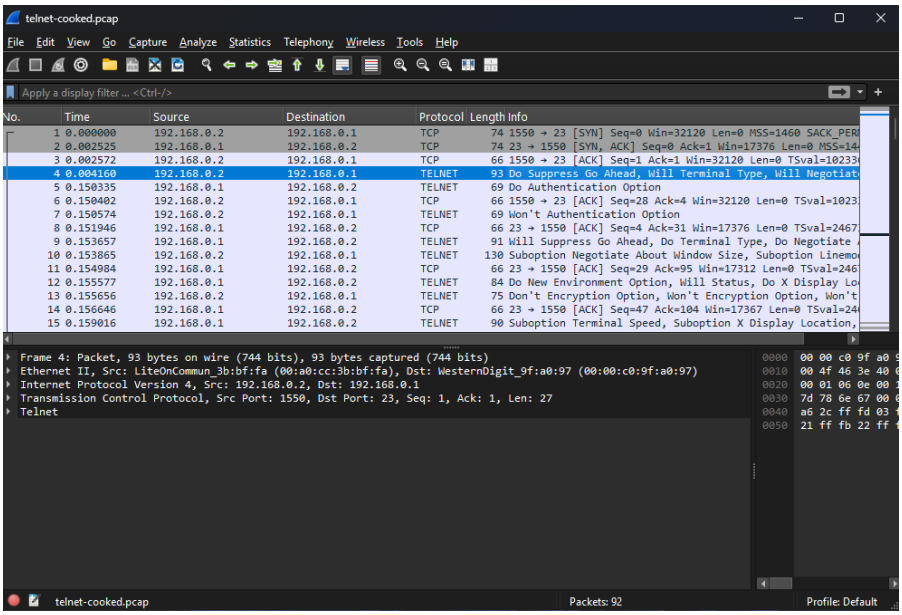


Figure 2. Packet list: the TCP handshake and the first Telnet option packets.

Step 2. The terminal details and the login

I used Follow TCP Stream to read the session as text (Figure 3). The client reported a terminal speed of 9600, the display bam.zing.org:0.0 and the terminal type xterm-color. The server identified itself as OpenBSD/i386 and asked for a login. The client typed **fake**. The server asked for a Password, and the client typed **user**. After that, the server printed the last login time, a warning that no Kerberos tickets were issued, the system line OpenBSD 2.6-beta (OOF) #4 with a build date of 12 Oct 1999, and a welcome message.

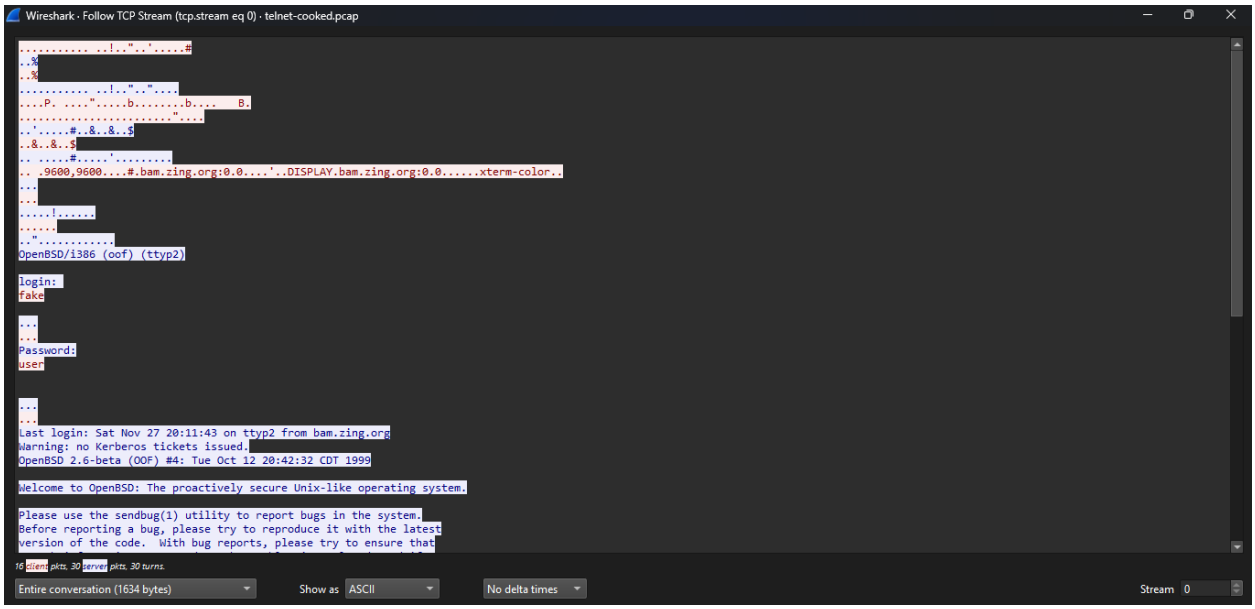


Figure 3. Follow TCP Stream, top of the session: terminal details, login name and password.

Step 3. The commands the user ran

At the shell prompt, the user ran `/sbin/ping www.yahoo.com`. The ping went to 204.71.200.67, and all 6 packets came back, with 0% packet loss and a round trip of about 70 to 75 ms. The user then ran `ls`, which printed nothing visible, and `ls -a`, which listed .cshrc, .login, .mailrc, .profile and .rhosts. The last command was `exit`, which ended the session (Figure 4).

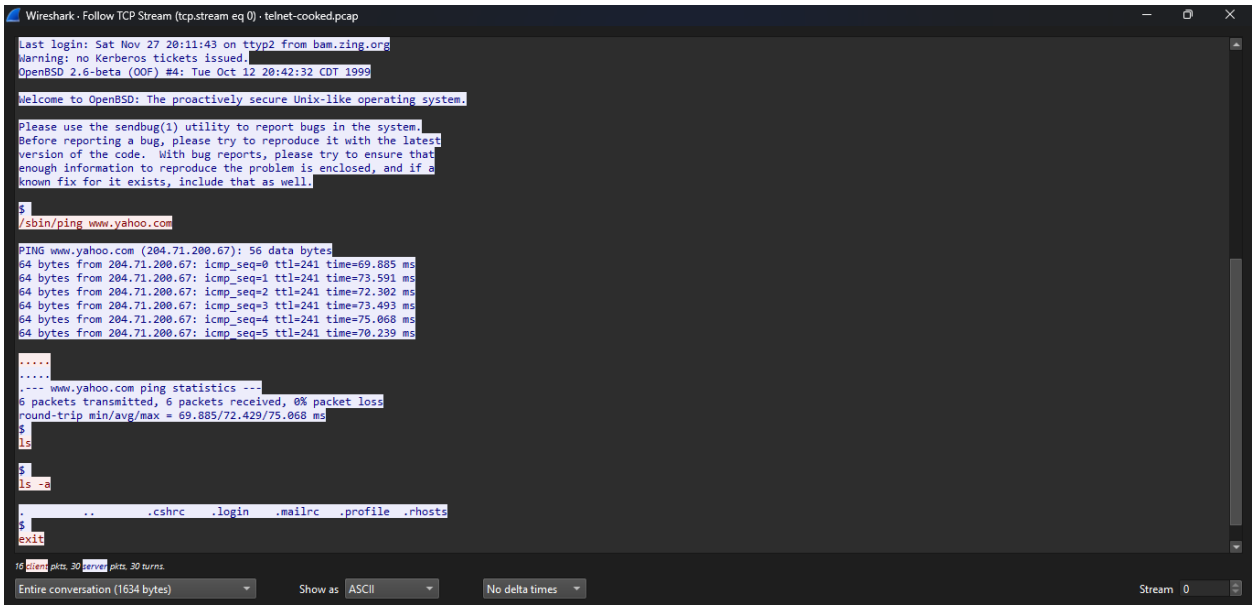


Figure 4. Follow TCP Stream, end of the session: ping, ls, ls -a and exit.

Step 4. The stream in numbers

The stream window shows 16 client packets and 30 server packets, and 1634 bytes in total. This matches the 1634 bytes of Telnet data in the Protocol Hierarchy.

4. Anything suspicious?

Result: I found no sign of an attack. It looks like a normal user session that was recorded in plain text. I noted five points:

- **The login name and the password are readable.** Telnet is not encrypted, so anyone who captures this traffic can read the login *fake* and the password *user*. These look like test values and not real credentials. This is the main weakness of Telnet.

- **The whole session is readable.** The commands, their output and the file names are all plain text. The listing shows a .rhosts file. This type of file can make a system trust other computers for remote logins, so I would check it in a real investigation.
- **No authentication or encryption option was agreed.** The client refused both options during the Telnet negotiation, as I described in Step 1.
- **Old system.** The server runs OpenBSD 2.6-beta from 1999. A system this old would be a risk on a network today.
- **One malformed packet.** Packet 72 goes from the server to the client. It is 67 bytes long and carries only 1 byte of Telnet data. Wireshark marks it as Malformed Packet (Exception occurred) and gives no more detail (Figure 5). The hex view shows that the data byte is ff. In Telnet, ff starts a control command, and it should be followed by a command byte. So I believe this is an incomplete command, probably an artefact of how the sample was recorded. Wireshark does not confirm this, so it is my reading. It is one tiny packet in a normal session, so I do not treat it as an attack.

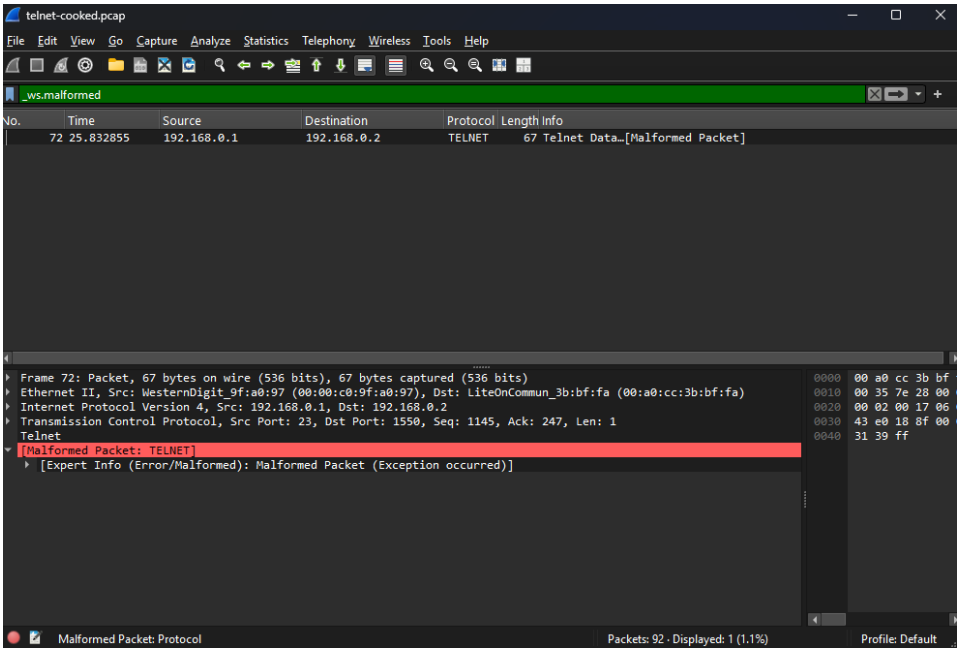


Figure 5. Filter _ws.malformed shows packet 72, with the Expert Info opened.

5. Conclusion

A user at 192.168.0.2 logged in to the server 192.168.0.1 over Telnet, ran a ping and two file listings, and logged out. The only protocols are Ethernet, IPv4, TCP and Telnet. I found nothing that points to an attack. The packets prove that Telnet is not safe for logins, because the password can be read by anyone who captures the traffic.

What I learned: Telnet should be replaced by SSH, which encrypts the login and the commands. Follow TCP Stream turns packets into a readable story. Wireshark labels a packet as malformed when it cannot read it fully, so I should look at the bytes before judging it.

Evidence: Figures 1 to 5 are screenshots I took from this capture in Wireshark. I downloaded the sample capture from the Wireshark SampleCaptures wiki.